

System Architecture

A systematic architecture blueprint that connects product workflow, application services, data ownership, asynchronous processing, and operational controls.

Version 1.0 | Made by DEX

Prepared from the Project VISTA source brief

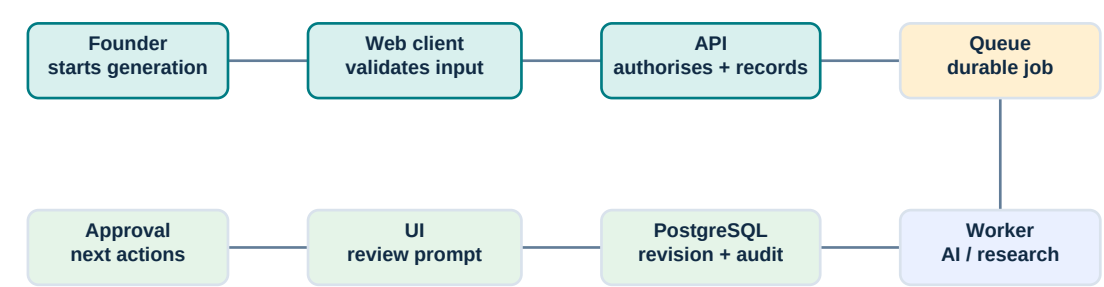
ARCHITECTURE

A coherent operating blueprint for an intelligent startup workspace.

1. Architecture at a Glance

VentureNest is organised around a project workspace. A responsive web client calls a secure API, which enforces permissions and coordinates domain actions. Long-running AI, research, and export jobs move through a durable queue. PostgreSQL remains the source of truth; object storage holds rendered exports and media.

2. End-to-End Request Flow



Generation produces a draft revision. Only the user-approved version drives downstream execution or exports.

3. Domain Boundaries

Boundary	Owns	Publishes events
Identity & access	Users, organisations, memberships, roles, sessions.	member.invited, member.role_changed
Project workspace	Projects, project context, settings, collaborators.	project.created, project.updated
Artifact studio	Artifacts, revisions, approval workflow, exports.	artifact.generated, artifact.approved
Research	Questions, jobs, sources, claims, competitor records.	research.completed, claim.verified
Financial planning	Models, assumptions, scenarios, outputs.	scenario.recalculated
Execution	Tasks, roadmap milestones, automation rules.	task.created, task.completed
Platform	Notifications, files, audit log, metering, integrations.	notification.sent, export.ready

4. Data Ownership and Integration Rules

- Each write goes through its owning domain API; domains do not update one another's tables directly.
- Use an outbox table with the same transaction as the domain write, then publish events asynchronously to avoid lost events.
- Derived search indexes, analytics, notifications, and exports consume events and are rebuildable.
- External LLM, search, email, calendar, and payment providers are accessed through adapters with timeouts, quotas, and observability.

- Public integration APIs use versioned routes and signed webhooks; do not expose internal event formats as public contracts.

5. Deployment Topology

Layer	Recommended implementation	Scaling concern
Edge	CDN, WAF, TLS termination, rate limiting.	Cache static assets; protect auth and generation endpoints.
Application	Stateless Next.js and FastAPI containers.	Scale on request latency and concurrent sessions.
Async	Worker containers consuming Redis/SQS-compatible queue.	Scale by queue depth; separate expensive research/export queues.
Data	Managed PostgreSQL, Redis, object storage.	Backups, failover, storage lifecycle, connection pooling.
Operations	CI/CD, secret manager, logging, metrics, tracing, alerting.	Progressive releases and rollback paths.

6. Resilience and Security Controls

Risk	Control
Provider timeout or bad response	Timeout, retry with backoff, circuit breaker, schema validation, persisted job failure state.
Duplicate job delivery	Idempotency key and transactional outbox; workers perform idempotent writes.
Cross-tenant access	Organisation/project scoping in query layer; tests for every permission boundary; optional database RLS.
Prompt injection from sources	Treat retrieved content as untrusted data, isolate it from instructions, filter and label sources.
Data loss	Point-in-time recovery, tested backups, object versioning, exportable customer data.
Uncontrolled AI spend	Per-user/tenant limits, model routing, budgets, cost telemetry, and admin alerts.

7. Recommended Evolution Path

Stage	Architecture decision
MVP	Modular monolith API plus separate worker; managed database; provider adapters; event outbox.
Growth	Split research/export worker pools, add read replica/search index, introduce feature flags and stronger analytics.
Scale	Extract high-volume domains only with clear ownership and operational need; adopt event stream and dedicated integration gateway.

8. Architecture Decision Record (ADR) Starters

- ADR-001: Choose FastAPI for AI orchestration and typed domain APIs.
- ADR-002: Use PostgreSQL plus JSONB rather than a separate document database for the MVP.
- ADR-003: Store approved artifact revisions immutably and make approval explicit.
- ADR-004: Run AI/research/export operations asynchronously with durable jobs.
- ADR-005: Adopt a provider-adapter layer for LLMs and external research sources.